



SAI RAM ENGINEERING COLLEGE

An Autonomous Institution | Affiliated to Anna University & Approved by AICTE, New Delhi
Accredited by NBA and NAAC "A+" | BIS/EOMS ISO 21001 : 2018 Certified and NIRF ranked institution
Sai Leo Nagar, West Tambaram, Chennai - 600 044. www.sairam.edu.in



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (Artificial Intelligence & Machine Learning)

LAB MANUAL

Academic Year : 2026 – 2027 (EVEN)

Year / Sem : III / V

Course Code / Title : Artificial Intelligence for Human Language Understanding Lab

List of Programs

1. a. Write a python program to find all unigrams, bigrams and trigrams present in the given corpus.
b. Implement a bigram model with Add-One smoothing and calculate the probability of the sentence “This is my cat” using Python.
2. a. Write a python program to perform tokenization by word and sentence using NLTK.
b. Write a python program to eliminate stop words using NLTK.
c. Write a python program to perform stemming and lemmatization using NLTK.
3. a. Write a python program to perform Parts of Speech tagging using NLTK.
b. Write a python program for chunking using NLTK.
4. a. Write a python program to perform Named Entity Recognition using NLTK.
b. Write a python program to find Term Frequency and Inverse Document Frequency (TF - IDF).
5. Write a python program for CYK parsing (Cocke Younger-Kasami Parsing) or Chart Parsing.
6. Write a Python program to train CBOW and Skip-gram Word2Vec models on a given text corpus.
7. Analyze word relationships by finding similar words and visualizing embeddings.
8. Write the python code to perform sentiment analysis using NLP.
9. Write the python code to detect Fake News using NLP.
10. Write a Python program to implement a language translation system that translates text from one language to another using any NLP library.
11. Develop a Python program to implement a next word prediction model using a given text corpus.
12. Develop a Python program to build a spam filter using NLP techniques.
13. Develop a Python program to design and implement a chatbot using NLP techniques.

TOTAL: 45 PERIODS

1. N-grams and Big ram Model with Add-One Smoothing:

Aim:

To write a Python Program to generate unigrams, bigrams and trigrams and compute sentence probability using a bigram model.

Algorithm/Procedure:

1. Start
2. Import the required NLTK libraries:
 - a. nltk
 - b. ngrams from nltk.util
 - c. word_tokenize from nltk.tokenize
3. Download the required tokenizer resources:
 - a. punkt
 - b. punkt_tab
4. Store the input sentence in a variable text.
5. Convert the text to lowercase using text.lower().
6. Tokenize the text into individual words using word_tokenize().
7. Generate unigrams by creating 1-word sequences from the token list using ngrams(tokens, 1).
8. Generate bigrams by creating 2-word sequences from the token list using ngrams(tokens, 2).
9. Generate trigrams by creating 3-word sequences from the token list using ngrams(tokens, 3).
10. Convert the generated n-grams into lists.
11. Display the unigrams, bigrams and trigrams
12. Stop

Program:

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab') # Required in newer NLTK versions
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
text = 'this is my cat this is my dog'
tokens = word_tokenize(text.lower())
print("Unigrams:", list(ngrams(tokens, 1)))
print("Bigrams:", list(ngrams(tokens, 2)))
print("Trigrams:", list(ngrams(tokens, 3)))
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
Unigrams: [('this',), ('is',), ('my',), ('cat',), ('this',), ('is',), ('my',), ('dog',)]
Bigrams: [('this', 'is'), ('is', 'my'), ('my', 'cat'), ('cat', 'this'), ('this', 'is'), ('is', 'my'), ('my', 'dog')]
Trigrams: [('this', 'is', 'my'), ('is', 'my', 'cat'), ('my', 'cat', 'this'), ('cat', 'this', 'is'), ('this', 'is', 'my'), ('is', 'my', 'dog')]
```

Result:

Thus, the program was implemented successfully and the output was verified.

2. Tokenization, Stopword Removal, Stemming and Lemmatization

Aim:

To write a Python Program to perform preprocessing tasks using NLTK.

Algorithm/Procedure:

1. Start
2. Import Libraries:
 - a. `nltk`
 - b. `word_tokenize`, `sent_tokenize` from `nltk.tokenize`
 - c. `stopwords` from `nltk.corpus`
 - d. `PorterStemmer`, `WordNetLemmatizer` from `nltk.stem`
3. Download Required NLTK Resources:
 - a. `punkt` (tokenizers)
 - b. `punkt_tab` (tokenizer update)
 - c. `stopwords` (common stopwords)
 - d. `wordnet` (lexical database for lemmatization)
 - e. `omw-1.4` (wordnet data)
4. Input Text:
 - a. Store the text in a variable `text`.
5. Sentence Tokenization:
 - a. Use `sent_tokenize(text)` to split the text into sentences.
6. Word Tokenization:
 - a. Use `word_tokenize(text)` to split the text into individual words.
7. Stopword Removal:
 - a. Get the set of English stopwords using `stopwords.words('english')`.
 - b. Remove words from the token list that appear in the stopwords set.
8. Stemming:
 - a. Initialize `PorterStemmer()`.
 - b. Apply the stemmer to each word in the filtered list.
9. Lemmatization:
 - a. Initialize `WordNetLemmatizer()`.
10. Apply the lemmatizer to each word in the filtered list.
11. Display Results:
12. Print sentence tokens ,word tokens ,words after stopwords removal ,stemmed words and lemmatized words
13. Stop

Program:

```
import nltk
# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
# Input Text
```

```
text = ""
```

```
Natural Language Processing is a branch of Artificial Intelligence.
```

```
It enables computers to understand human language and process text efficiently.
```

```
""
```

```
# Sentence Tokenization
```

```
sentences = sent_tokenize(text)
```

```
# Word Tokenization
```

```
words = word_tokenize(text)
```

```
# Stopword Removal
```

```
stop_words = set(stopwords.words('english'))
```

```
filtered_words = [word for word in words if word.lower() not in stop_words]
```

```
# Stemming
```

```
stemmer = PorterStemmer()
```

```
stemmed_words = [stemmer.stem(word) for word in filtered_words]
```

```
# Lemmatization
```

```
lemmatizer = WordNetLemmatizer()
```

```
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]
```

```
# Display Results
```

```
print("Sentence Tokens:")
```

```
print(sentences)
```

```
print("\nWord Tokens:")
```

```
print(words)
```

```
print("\nWords After Stopword Removal:")
```

```
print(filtered_words)
```

```
print("\nStemmed Words:")
```

```
print(stemmed_words)
```

```
print("\nLemmatized Words:")
```

```
print(lemmatized_words)
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
```

```
[nltk_data] Package punkt is already up-to-date!
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
```

```
[nltk_data] Package punkt_tab is already up-to-date!
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
```

```
[nltk_data] Unzipping corpora/stopwords.zip.
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

```
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
```

```
Sentence Tokens:
```

```
['Natural Language Processing is a branch of Artificial Intelligence.', 'It enables computers to understand human language and process text efficiently.']
```

```
Word Tokens:
```

```
['Natural', 'Language', 'Processing', 'is', 'a', 'branch', 'of', 'Artificial', 'Intelligence', '.', 'It', 'enables', 'computers', 'to', 'understand', 'human', 'language', 'and', 'process', 'text', 'efficiently', '.']
```

```
Words After Stopword Removal:
```

```
['Natural', 'Language', 'Processing', 'branch', 'Artificial', 'Intelligence', '.', 'enables', 'computers', 'understand', 'human', 'language', 'process', 'text', 'efficiently', '.']
```

```
Stemmed Words:
```

```
['natur', 'languag', 'process', 'branch', 'artifici', 'intellig', '.', 'enabl', 'comput', 'understand', 'human', 'languag', 'process', 'text', 'effici', '.']
```

```
Lemmatized Words:
```

['Natural', 'Language', 'Processing', 'branch', 'Artificial', 'Intelligence', '.', 'enables', 'computer', 'understand', 'human', 'language', 'process', 'text', 'efficiently', '.']

Result:

Thus, the program was implemented successfully and the output was verified.

3. POS Tagging and Chunking

Aim:

To write a Python Program to identify grammatical categories and phrase structures.

Algorithm/Procedure:

1. Start
2. Import the required NLTK libraries and download the necessary resources (punkt, punkt_tab, averaged_perceptron_tagger, averaged_perceptron_tagger_eng).
3. Store the input sentence in a variable.
4. Tokenize the sentence into words using `word_tokenize()`.
5. Perform Part-of-Speech (POS) tagging on the tokenized words using `pos_tag()`.
6. Define the chunk grammar for noun phrases:
 - a. NP: {<DT>?<JJ>*<NN>}
7. Create a chunk parser using `RegexParser()` with the defined grammar.
8. Parse the POS-tagged words to generate a chunk tree.
9. Display the POS tags and chunk tree.
10. Traverse the chunk tree and identify subtrees labeled NP.
11. Extract and print the noun phrases from the identified chunks.
12. Stop.

Program:

```
import nltk

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexParser

# Input sentence
sentence = "The little cat sat on the red mat"

# Tokenization
words = word_tokenize(sentence)

# POS Tagging
pos_tags = pos_tag(words)

print("POS Tags:")
print(pos_tags)

# Define Chunk Grammar
grammar = r"""
NP: {<DT>?<JJ>*<NN>}
"""

# Create Chunk Parser
chunk_parser = RegexParser(grammar)

# Perform Chunking
```

```

chunk_tree = chunk_parser.parse(pos_tags)

print("\nChunk Tree:")
print(chunk_tree)

# Display Chunked Phrases
print("\nNoun Phrases:")
for subtree in chunk_tree.subtrees():
    if subtree.label() == 'NP':
        print(" ".join(word for word, tag in subtree.leaves()))

```

Output:

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...

```

POS Tags:

```
[('The', 'DT'), ('little', 'JJ'), ('cat', 'NN'), ('sat', 'VBD'), ('on', 'IN'), ('the', 'DT'), ('red', 'JJ'), ('mat', 'NN')]
```

Chunk Tree:

```

(S
  (NP The/DT little/JJ cat/NN)
  sat/VBD
  on/IN
  (NP the/DT red/JJ mat/NN))

```

Noun Phrases:

The little cat

the red mat

```
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

Result:

Thus, the program, was implemented successfully and the output was verified.

4. Named Entity Recognition and TF-IDF

Aim:

To write a program to identify named entities and compute TF-IDF scores.

Algorithm/Procedure:

1. Start
2. Import required libraries.
3. Download necessary NLTK resources.
4. Input the text.
5. Tokenize the text into words.
6. Perform Part-of-Speech (POS) tagging.
7. Apply Named Entity Recognition using `ne_chunk()`.
8. Display the identified named entities.
9. Stop.

Program:

```
import nltk
# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
from nltk import word_tokenize, pos_tag, ne_chunk
from sklearn.feature_extraction.text import TfidfVectorizer
# -----
# Part A: Named Entity Recognition
# -----
text = "Barack Obama was born in Hawaii and worked at Microsoft."
tokens = word_tokenize(text)
pos_tags = pos_tag(tokens)
print("POS Tagged Words:")
print(pos_tags)
print("\nNamed Entities:")
ner_tree = ne_chunk(pos_tags)
print(ner_tree)
# -----
# Part B: TF-IDF Calculation
# -----
documents = [
    "Natural Language Processing is interesting",
    "Machine Learning is a part of Artificial Intelligence",
    "Natural Language Processing uses Machine Learning"
]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)
feature_names = vectorizer.get_feature_names_out()
print("\nTF-IDF Feature Names:")
print(feature_names)
print("\nTF-IDF Matrix:")
print(tfidf_matrix.toarray())
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker.zip.
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker_tab.zip.
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Unzipping corpora/words.zip.
```

POS Tagged Words:

```
[('Barack', 'NNP'), ('Obama', 'NNP'), ('was', 'VBD'), ('born', 'VBN'), ('in', 'IN'), ('Hawaii', 'NNP'), ('and', 'CC'), ('worked', 'VBD'), ('at', 'IN'), ('Microsoft', 'NNP'), ('.', '.')] ]
```

Named Entities:

```
(S
 (PERSON Barack/NNP)
 (PERSON Obama/NNP)
 was/VBD
 born/VBN
 in/IN
 (GPE Hawaii/NNP)
 and/CC
 worked/VBD
 at/IN
 (ORGANIZATION Microsoft/NNP)
 ./.)
```

TF-IDF Feature Names:

```
['artificial' 'intelligence' 'interesting' 'is' 'language' 'learning'
 'machine' 'natural' 'of' 'part' 'processing' 'uses']
```

TF-IDF Matrix:

```
[[0.      0.      0.54935123 0.41779577 0.41779577 0.
  0.      0.41779577 0.      0.      0.41779577 0.      ]
 [0.41756662 0.41756662 0.      0.31757018 0.      0.31757018
  0.31757018 0.      0.41756662 0.41756662 0.      0.      ]
 [0.      0.      0.      0.      0.38550292 0.38550292
```

0.38550292 0.38550292 0. 0. 0.38550292 0.50689001]]

Result:

Thus, the program was implemented successfully and the output was verified.

5. CYK Parsing / Chart Parsing

Aim:

To write a Python program to parse sentences using context-free grammar.

Algorithm/Procedure:

1. Start.
2. Import the required NLTK modules (CFG and ChartParser).
3. Define the Context-Free Grammar (CFG) with production rules.
4. Create a Chart Parser using the defined grammar.
5. Input the sentence to be parsed.
6. Tokenize the sentence into individual words.
7. Display the input sentence.
8. Pass the tokenized words to the parser.
9. Generate parse trees based on the grammar rules.
10. Display all valid parse trees.
11. Stop.

Program:

```
import nltk
from nltk import CFG
from nltk.parse import ChartParser
# Define Context-Free Grammar
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the'
N -> 'cat' | 'dog'
V -> 'chased' | 'saw'
""")
# Create Chart Parser
parser = ChartParser(grammar)
# Input Sentence
sentence = "the cat chased the dog"
# Tokenize
words = sentence.split()
print("Input Sentence:")
print(sentence)
print("\nParse Trees:")
# Parse and Display Trees
for tree in parser.parse(words):
    print(tree)
```

Output:

Input Sentence:

The cat chased the dog

Parse Trees:

(S (NP (Det the) (N cat)) (VP (V chased) (NP (Det the) (N dog))))

Result:

Thus, the program was implemented successfully and the output was verified.

6. CBOW and Skip-Gram Word2Vec

Aim:

To write a Python program to train Word2Vec models and analyze word relationships.

Algorithm/Procedure:

1. Start.
2. Install and import the required libraries (`gensim`, `sklearn`, and `matplotlib`).
3. Create a sample text corpus consisting of multiple sentences.
4. Train a Word2Vec model using the **CBOW (Continuous Bag of Words)** architecture (`sg = 0`).
5. Train another Word2Vec model using the **Skip-Gram** architecture (`sg = 1`).
6. Retrieve the words most similar to the target word ("machine") from the CBOW model.
7. Display the similar words obtained from the CBOW model.
8. Retrieve the words most similar to the target word ("machine") from the Skip-Gram model.
9. Display the similar words obtained from the Skip-Gram model.
10. Extract the vocabulary words and their corresponding word vectors from the Skip-Gram model.
11. Apply Principal Component Analysis (PCA) to reduce the word vector dimensions from 50 to 2.
12. Transform the word vectors into two-dimensional coordinates.
13. Create a scatter plot for the reduced word vectors.
14. Annotate each point with its corresponding word.
15. Display the Word2Vec embedding visualization.
16. Stop.

Program:

```
!pip install -q gensim
from gensim.models import Word2Vec
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
# Sample Corpus
corpus = [
    ["natural", "language", "processing", "is", "interesting"],
    ["machine", "learning", "is", "a", "part", "of", "artificial", "intelligence"],
    ["deep", "learning", "improves", "machine", "learning"],
    ["natural", "language", "processing", "uses", "machine", "learning"],
    ["artificial", "intelligence", "includes", "machine", "learning"],
    ["cats", "and", "dogs", "are", "pets"],
    ["dog", "is", "a", "faithful", "animal"],
    ["cat", "is", "a", "cute", "animal"]
]
# -----
# CBOW Model
# -----
cbow_model = Word2Vec(
    sentences=corpus,
    vector_size=50,
    window=3,
    min_count=1,
    sg=0 # CBOW
)
# -----
# Skip-Gram Model
# -----
skipgram_model = Word2Vec(
    sentences=corpus,
```

```

    vector_size=50,
    window=3,
    min_count=1,
    sg=1 # Skip-Gram
)
# Similar Words
print("CBOW - Similar words to 'machine':")
print(cbow_model.wv.most_similar('machine'))
print("\nSkip-Gram - Similar words to 'machine':")
print(skipgram_model.wv.most_similar('machine'))
# -----
# Embedding Visualization
# -----
words = list(skipgram_model.wv.index_to_key)
vectors = [skipgram_model.wv[word] for word in words]
# PCA Reduction
pca = PCA(n_components=2)
result = pca.fit_transform(vectors)
# Plot
plt.figure(figsize=(8,6))
for i, word in enumerate(words):
    plt.scatter(result[i,0], result[i,1])
    plt.annotate(word,
        xy=(result[i,0], result[i,1]))
plt.title("Word2Vec Embeddings (Skip-Gram)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
Output:

```

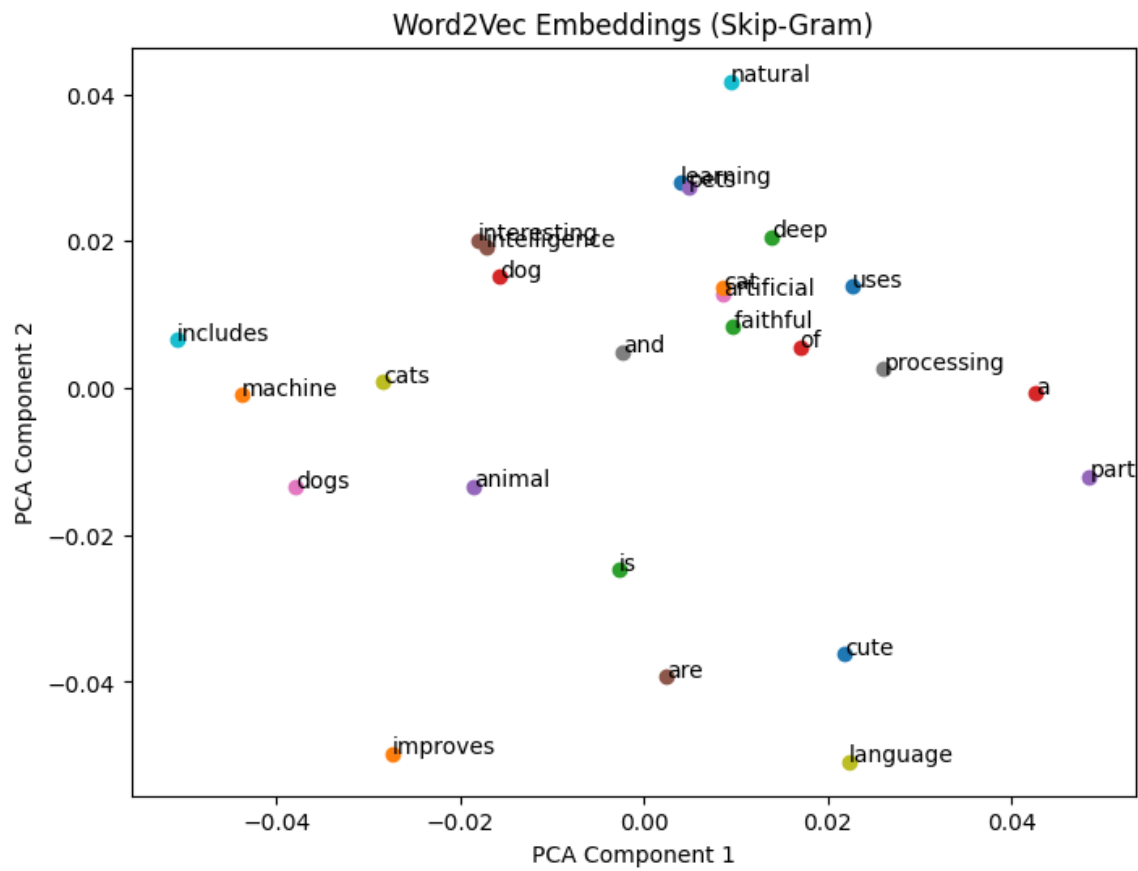
27.9/27.9 MB 40.4 MB/s eta 0:00:00

CBOW - Similar words to 'machine':

```
[('dogs', 0.22986409068107605), ('interesting', 0.21900270879268646), ('includes',
0.1608886569738388), ('cats', 0.1486944556236267), ('animal', 0.12495838850736618), ('faithful',
0.0807289257645607), ('artificial', 0.07405998557806015), ('improves', 0.05498278886079788),
('learning', 0.04266327992081642), ('intelligence', 0.018151162192225456)]
```

Skip-Gram - Similar words to 'machine':

```
[('dogs', 0.23028233647346497), ('interesting', 0.21907445788383484), ('includes',
0.1608043760061264), ('cats', 0.14878608286380768), ('animal', 0.12491549551486969), ('faithful',
0.08065251260995865), ('artificial', 0.07397226989269257), ('improves', 0.05522400885820389),
('learning', 0.042760320007801056), ('intelligence', 0.018298432230949402)]
```



Result:

Thus, the program was implemented successfully and the output was verified.

7. Sentiment Analysis

Aim:

To write a Python program to determine sentiment polarity of text.

Algorithm/Procedure:

1. Start.
2. Import the `TextBlob` library.
3. Define a function `analyze_sentiment(text)`.
4. Create a `TextBlob` object from the input text.
5. Extract the polarity score from the sentiment analysis result.
6. Extract the subjectivity score from the sentiment analysis result.
7. Determine the sentiment category:
 - a. If $\text{polarity} > 0$, classify as **Positive**.
 - b. If $\text{polarity} < 0$, classify as **Negative**.
 - c. If $\text{polarity} = 0$, classify as **Neutral**.
8. Display the input text.
9. Display the polarity score.
10. Display the subjectivity score.
11. Display the sentiment category.
12. Interpret the sentiment based on the polarity value.
13. Interpret the text as subjective or objective based on the subjectivity value.
14. Accept a sentence or paragraph as input from the user.
15. Call the `analyze_sentiment()` function with the user input.
16. Display the sentiment analysis report.
17. Stop.

Program:

```
# Install TextBlob if not already installed
# pip install textblob
# Import required library
from textblob import TextBlob
# Function to perform sentiment analysis
def analyze_sentiment(text):
    # Create TextBlob object
    blob = TextBlob(text)
    # Extract sentiment scores
    polarity = blob.sentiment.polarity
    subjectivity = blob.sentiment.subjectivity
    # Determine sentiment category
    if polarity > 0:
        sentiment = "POSITIVE"
    elif polarity < 0:
        sentiment = "NEGATIVE"
    else:
        sentiment = "NEUTRAL"
    # Display Results
    print("\n=====")
    print("SENTIMENT ANALYSIS REPORT")
    print("=====")
    print("\nInput Text:")
    print(text)
    print("\nPolarity Score :", polarity)
    print("Subjectivity Score :", subjectivity)
```

```

print("Sentiment :", sentiment)
# Additional Interpretation
print("\nInterpretation:")
if polarity > 0:
    print("The text expresses a positive opinion or emotion.")
elif polarity < 0:
    print("The text expresses a negative opinion or emotion.")
else:
    print("The text does not express a strong positive or negative opinion.")
if subjectivity > 0.5:
    print("The text is highly subjective and opinion-based.")
else:
    print("The text is relatively objective and fact-based.")
print("=====")
# Main Program
print("=====")
print("    SENTIMENT ANALYSIS USING NLP")
print("=====")
# User Input
text = input("\nEnter a sentence or paragraph:\n")
# Call Function
analyze_sentiment(text)

```

Output:

```
=====
```

SENTIMENT ANALYSIS USING NLP

```
=====
```

Enter a sentence or paragraph:

I really enjoy learning Natural Language Processing. It is very interesting and useful.

```
=====
```

SENTIMENT ANALYSIS REPORT

```
=====
```

Input Text:

I really enjoy learning Natural Language Processing. It is very interesting and useful.

Polarity Score : 0.3625

Subjectivity Score : 0.3875

Sentiment : POSITIVE

Interpretation:

The text expresses a positive opinion or emotion.

The text is relatively objective and fact-based.

```
=====
```

```
=====
```

SENTIMENT ANALYSIS USING NLP

```
=====
```

Enter a sentence or paragraph:

The movie was boring and disappointing.

```
=====
```

SENTIMENT ANALYSIS REPORT

```
=====
```

Input Text:

The movie was boring and disappointing.

Polarity Score : -0.8

Subjectivity Score : 0.85

Sentiment : NEGATIVE

Interpretation:

The text expresses a negative opinion or emotion.

The text is highly subjective and opinion-based.

=====

Result:

Thus, the program was implemented successfully and the output was verified.

8. Fake News Detection Using TF-IDF and Logistic Regression

Aim:

To write a Python program to classify news as real or fake using TF-IDF and Logistic Regression

Algorithm/Procedure:

1. Start.
2. Import the required libraries (`pandas`, `TfidfVectorizer`, `train_test_split`, `LogisticRegression`, and evaluation metrics).
3. Read the CSV dataset path from the user.
4. Load the dataset into a `DataFrame`.
5. Display a preview of the dataset.
6. Check whether the dataset contains the required columns (`text` and `label`).
7. Extract the text data as features and labels as target values.
8. Initialize the TF-IDF Vectorizer with appropriate parameters.
9. Convert the text data into TF-IDF feature vectors.
10. Split the dataset into training and testing sets.
11. Initialize the Logistic Regression classifier.
12. Train the classifier using the training data.
13. Predict labels for the testing data.
14. Calculate the model accuracy.
15. Generate the classification report.
16. Display the accuracy and classification report.
17. Accept a news article from the user.
18. Check whether the user entered "exit".
19. If "exit" is entered, terminate the prediction process.
20. Otherwise, convert the news article into a TF-IDF vector.
21. Predict whether the news is Real or Fake using the trained model.
22. Display the prediction result.
23. Repeat Steps 17–22 until the user enters "exit".
24. Stop.

Program:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Read Dataset from User
file_path = input("Enter the path of the CSV dataset: ")

# Load Dataset
df = pd.read_csv(file_path)
print("\nDataset Preview:")
print(df.head())

# Check Required Columns
if 'text' not in df.columns or 'label' not in df.columns:
    print("\nDataset must contain 'text' and 'label' columns.")
    exit()

# Features and Labels
X = df['text']
y = df['label']

# Convert Text into TF-IDF Features
vectorizer = TfidfVectorizer(
```

```

    stop_words='english',
    max_df=0.7
)

X_tfidf = vectorizer.fit_transform(X)
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf,
    y,
    test_size=0.2,
    random_state=42
)
# Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# Model Evaluation
y_pred = model.predict(X_test)
print("\nModel Accuracy:")
print(round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Predict User News
while True:
    news = input("\nEnter a news article to check (or type 'exit'): ")
    if news.lower() == "exit":
        break
    news_vector = vectorizer.transform([news])
    prediction = model.predict(news_vector)
    print("\nPrediction:")
    if prediction[0].upper() == "FAKE":
        print("FAKE NEWS")
    else:
        print("REAL NEWS")

```

Output:

Result:

Thus, the program was implemented successfully and the output was verified.

9. Language Translation Using Google Translator

Aim:

To write a Python program to translate text between languages using Google Translator.

Algorithm/Procedure:

1. Start.
2. Install and import the required library (`deep_translator`).
3. Display the program title.
4. Accept the text to be translated from the user.
5. Accept the source language and the target language code from the user.
6. Initialize the Google Translator with the specified source and target languages.
7. Translate the input text using the translator and Store the translated text.
8. Display the original text and the translated text.
9. Stop.

Program:

```
!pip install deep-translator
from deep_translator import GoogleTranslator
print("=====")
print("  LANGUAGE TRANSLATION")
print("=====")
# Input text
text = input("Enter text to translate: ")
# Source language
source_lang = input(
    "Enter source language code (e.g., en, ta, fr, hi): "
)
# Target language
target_lang = input(
    "Enter target language code (e.g., en, ta, fr, hi): "
)
```

Translation

```
translated_text = GoogleTranslator(
    source=source_lang,
    target=target_lang
).translate(text)
print("\nOriginal Text:")
print(text)
print("\nTranslated Text:")
print(translated_text)
```

Output:

Requirement already satisfied: deep-translator in /usr/local/lib/python3.12/dist-packages (1.11.4)
Requirement already satisfied: beautifulsoup4<5.0.0,>=4.9.1 in /usr/local/lib/python3.12/dist-packages (from deep-translator) (4.13.5)
Requirement already satisfied: requests<3.0.0,>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from deep-translator) (2.32.4)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4<5.0.0,>=4.9.1->deep-translator) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4<5.0.0,>=4.9.1->deep-translator) (4.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.23.0->deep-translator) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.23.0->deep-translator) (3.15)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.23.0->deep-translator) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.23.0->deep-translator) (2026.5.20)

=====

LANGUAGE TRANSLATION

=====

Enter text to translate: how r u

Enter source language code (e.g., en, ta, fr, hi): en

Enter target language code (e.g., en, ta, fr, hi): ta

Original Text:

how r u

Translated Text:

□□□□□□ □□□□□□□□□□□□

Result:

Thus, the program was implemented successfully and the output was verified.

10. Next Word Prediction

Aim:

To write a Python program to predict the next word from a corpus.

Algorithm/Procedure:

1. Start.
2. Import the required libraries (`nltk`, `bigrams`, `defaultdict`, and `Counter`).
3. Download the necessary NLTK tokenization resources.
4. Define the sample text corpus.
5. Convert the corpus to lowercase.
6. Tokenize the corpus into individual words.
7. Generate all bigram pairs from the tokenized words.
8. Create an empty dictionary to store word relationships.
9. For each bigram pair, store the second word as a possible successor of the first word.
10. Accept an input word from the user.
11. Convert the input word to lowercase.
12. Check whether the input word exists in the bigram dictionary.
13. If the word exists:
 - o Retrieve all possible next words.
 - o Count the frequency of each next word.
 - o Select the most frequently occurring next word as the prediction.
 - o Display the possible next words.
 - o Display the predicted next word.
14. Otherwise, display a message indicating that the word is not found in the corpus.
15. Stop.

Program:

```
import nltk
from nltk.util import bigrams
from collections import defaultdict, Counter
# Download NLTK resources (run once)
nltk.download('punkt')
nltk.download('punkt_tab')
# Sample Corpus
corpus = """
Natural Language Processing is an important field of Artificial Intelligence.
Natural Language Processing enables computers to understand human language.
Artificial Intelligence and Machine Learning are transforming technology.
Machine Learning is a subset of Artificial Intelligence.
"""
# Text Preprocessing
tokens = nltk.word_tokenize(corpus.lower())
# Generate Bigrams
bigram_pairs = list(bigrams(tokens))
# Create Bigram Dictionary
word_dict = defaultdict(list)
for word1, word2 in bigram_pairs:
    word_dict[word1].append(word2)
# User Input
input_word = input("Enter a word: ").lower()
# Predict Next Word
if input_word in word_dict:
```

```
next_words = word_dict[input_word]
prediction = Counter(next_words).most_common(1)[0][0]
print("\nPossible Next Words:")
print(next_words)
print("\nPredicted Next Word:")
print(prediction)
else:
    print("\nWord not found in corpus.")
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
Enter a word: transforming
Possible Next Words:
['technology']
Predicted Next Word:
technology
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
Enter a word: is
Possible Next Words:
['an', 'a']
Predicted Next Word:
an
```

Result:

Thus, the program was implemented successfully and the output was verified.

11. Spam Filter Using TF-IDF and Naive Bayes

Aim:

To write a Python program to classify messages as spam or ham.

Algorithm/Procedure:

1. Start.
2. Import the required libraries (pandas, TfidfVectorizer, train_test_split, MultinomialNB, and evaluation metrics).
3. Create a dataset containing messages and their corresponding labels (Spam/Ham).
4. Convert the dataset into a DataFrame.
5. Display the dataset preview.
6. Separate the message column as features (X).
7. Separate the label column as target values (y).
8. Initialize the TF-IDF Vectorizer.
9. Convert the text messages into TF-IDF feature vectors.
10. Split the dataset into training and testing sets.
11. Initialize the Multinomial Naive Bayes classifier.
12. Train the classifier using the training data.
13. Predict the labels for the testing data.
14. Calculate the model accuracy.
15. Generate the classification report.
16. Display the accuracy and classification report.
17. Accept a message from the user for classification.
18. Convert the input message into a TF-IDF vector.
19. Predict the class label using the trained model.
20. If the predicted label is "Spam", display SPAM MESSAGE.
21. Otherwise, display HAM MESSAGE.
22. Stop.

Program:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

Sample Dataset

```
data = {
    'message': [
        "Congratulations! You have won a free lottery ticket",
        "Claim your cash prize now",
        "Limited time offer, click here",
        "Meeting scheduled at 10 AM tomorrow",
        "Please submit your assignment today",
        "Let's have lunch together",
        "You won a free vacation package",
        "Your bank account has been credited"
    ],
    'label': [
        'Spam',
        'Spam',
        'Spam',
        'Ham',
        'Ham',
        'Ham',
        'Ham',
        'Ham'
    ]
}
```

```

        'Spam',
        'Ham'
    ]
}
df = pd.DataFrame(data)
print("Dataset Preview:")
print(df)
# Features and Labels
X = df['message']
y = df['label']
# TF-IDF Feature Extraction
vectorizer = TfidfVectorizer(
    stop_words='english'
)
X_tfidf = vectorizer.fit_transform(X)
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf,
    y,
    test_size=0.25,
    random_state=42
)
# Train Naive Bayes Classifier
model = MultinomialNB()
model.fit(X_train, y_train)
# Evaluate Model
y_pred = model.predict(X_test)
print("\nAccuracy:")
print(round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# User Input for Prediction
message = input("\nEnter a message to classify:\n")
message_vector = vectorizer.transform([message])
prediction = model.predict(message_vector)
print("\nPrediction:")
if prediction[0] == "Spam":
    print("SPAM MESSAGE")
else:
    print("HAM MESSAGE")

```

Output:

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.

```

```
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
```

Dataset Preview:

	message	label
0	Congratulations! You have won a free lottery t...	Spam
1	Claim your cash prize now	Spam
2	Limited time offer, click here	Spam
3	Meeting scheduled at 10 AM tomorrow	Ham
4	Please submit your assignment today	Ham
5	Let's have lunch together	Ham
6	You won a free vacation package	Spam
7	Your bank account has been credited	Ham

Accuracy:

50.0 %

Classification Report:

	precision	recall	f1-score	support
Ham	0.50	1.00	0.67	1
Spam	0.00	0.00	0.00	1
accuracy	0.50	2		
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

Enter a message to classify:

Let's have lunch together

Prediction:

HAM MESSAGE

Result:

Thus, the program was implemented successfully and the output was verified.

12. Chatbot using NLP

Aim:

To design a simple rule-based chatbot.

Algorithm/Procedure:

1. Start.
2. Import the required NLTK chatbot modules (`Chat` and `reflections`).
3. Define a list of patterns and corresponding responses.
4. Create a chatbot object using the defined patterns and reflections.
5. Display the chatbot welcome message.
6. Inform the user that typing "**bye**" will exit the chatbot.
7. Accept user input.
8. Compare the user input with the predefined patterns.
9. If a matching pattern is found, select and display the corresponding response.
10. If no specific pattern matches, display a default response.
11. Continue the conversation by accepting further user inputs.
12. If the user enters "**bye**" or "**goodbye**", display a farewell message.
13. Terminate the chatbot session.
14. Stop.

Program:

```
import nltk
from nltk.chat.util import Chat, reflections
# Define Patterns and Responses
pairs = [
    [
        r"hi|hello|hey",
        ["Hello! How can I help you today?",
         "Hi there! Nice to meet you.",
         "Hello! Welcome."]
    ],
    [
        r"what is your name ?",
        ["My name is NLP Chatbot.",
         "You can call me Chatbot."]
    ],
    [
        r"how are you ?",
        ["I am fine. Thank you for asking!",
         "I'm doing well."]
    ],
    [
        r"what can you do ?",
        ["I can answer simple questions and chat with you.",
         "I can help you learn NLP concepts."]
    ],
    [
        r"(.*) your creator ?",
        ["I was created using Python and NLTK."]
    ],
    [
```

```

    r"(.*) machine learning (.*)",
    ["Machine Learning is a branch of Artificial Intelligence."]
],
[
    r"(.*) nlp (.*)",
    ["NLP stands for Natural Language Processing."]
],

[
    r"bye|goodbye",
    ["Goodbye! Have a nice day.",
     "Bye! See you again."]
],

[
    r"(.*)",
    ["Sorry, I don't understand that.",
     "Could you please rephrase your question?"]
]

```

Create Chatbot

```

chatbot = Chat(pairs, reflections)
print("=====")
print("    NLP CHATBOT")
print("=====")
print("Type 'bye' to exit.\n")

```

Start Chat

```
chatbot.converse()
```

Output:

```

=====
    NLP CHATBOT
=====
Type 'bye' to exit.
>what is your name?
My name is NLP Chatbot.
>What can I do?
Could you please rephrase your question?
>what can you do ?
I can answer simple questions and chat with you.
>bye
Bye! See you again.

```

Result:

Thus, the program was implemented successfully and the output was verified.